BUS ROUTE MANAGEMENT SYSTEM

1. Changes Suggested in Updated Project

The previous project was a basic circular linked list program that could only add bus stops and display the route. To make the project more advanced, additional data structures and operations were introduced.

The following changes were suggested:

1. Circular Doubly Linked List: Supports adding and deleting bus stops and traversing the route in both forward and backward directions.

2. Addition and Deletion of Bus Stops: Allows users to insert new stops and remove existing stops from the route.

3. Searching Bus Stops: Helps users find a particular stop in the route.

4. Forward and Backward Route Display: Displays bus stops in both directions.

5. Next and Previous Stop: Displays the previous and next stops for a selected bus stop.

6. Stack: Stores recently added stops and supports undoing the last addition.

7. Queue: Manages pending bus stops using enqueue and dequeue operations.

These improvements make the project more interactive and demonstrate the practical applications of data structures in C++.

2. Problem Statement

The previous Bus Route Management System used a circular linked list to store bus stops and display them in sequence. However, it lacked essential features such as deleting stops, searching for a particular stop, displaying the route in reverse order, and managing pending route updates.

To overcome these limitations, the project is enhanced using a circular doubly linked list, stack, and queue. The system provides efficient bus stop management, route navigation, undo functionality, and queue-based operations.

3. Solution

The proposed solution is a C++ program that combines three data structures to improve bus route management.

Circular Doubly Linked List: Stores bus stop names and allows insertion, deletion, searching, and traversal in both directions.

Stack: Stores the names of newly added stops. It follows the Last In, First Out (LIFO) principle, allowing the most recent addition to be undone.

Queue: Stores pending bus stops. It follows the First In, First Out (FIFO) principle, allowing users to add stops to the queue and remove the earliest pending stop. A pending stop can also be processed and added to the main route.

The program uses a menu-driven interface so users can select the required operation.

4. Objectives

The main objectives of the project are:

1. To implement a circular doubly linked list for bus route management.
2. To insert and delete bus stops dynamically.
3. To search for a particular bus stop in the route.
4. To display the bus route in forward and backward directions.
5. To identify the next and previous stops for a selected stop.
6. To implement a stack for undoing the last addition.
7. To implement a queue for managing pending bus stops.
8. To understand the practical applications of linked lists, stacks, and queues in C++.
9. To improve the flexibility and efficiency of bus route operations.